Kenneth Poenadi

My project is an event-commerce platform that uses a client-server architecture and is structured as a full - stack web application with a React frontend, Go backend, and PostgreSQL database. The React frontend acts as the client, the Go backend acts as the server that handles business logic and API requests, and PostgreSQL is used to store the application data.

The application is divided into two main parts: the frontend and the backend. On the frontend, I used React with Vite and TypeScript. The frontend is organized into pages, reusable components, API modules, and type definitions. Pages handle the main user and admin screens, components are used for reusable UI elements, and API modules such as orderApi, productApi, bundleApi, and userApi handle communication with the backend. This structure helps keep the UI logic separated from the API request logic.

```
kennethpoenadi@Kenneths-MacBook-Air frontend % tree -L 3 -d
.
├── dist
│   ├── Footer
│   ├── Navbar
│   ├── assets
│   ├── background
│   └── fonts
├── public
│   ├── Footer
│   ├── Navbar
│   ├── assets
│   ├── background
│   └── fonts
└── src
    ├── api
    │   └── uploadthing
    ├── components
    │   ├── bundle
    │   └── product
    ├── hooks
    ├── lib
    │   └── constants
    ├── pages
    │   ├── admin
    │   ├── auth
    │   ├── cart
    │   ├── checkout
    │   ├── giveaway
    │   ├── home
    │   ├── notfound
    │   ├── products
    │   └── profile
    ├── routes
    ├── types
    └── utils

35 directories
```

On the backend, I used Go with the Gin framework. The backend is structured by domain, with separate routes for authentication, products, bundles, orders, users, uploads, reimbursements, and admin-related features. Each route is connected to controllers that handle the business logic, while database operations

are managed using GORM models. This makes the backend easier to maintain because each feature has its own responsibility and is not mixed with unrelated logic.

```
kennethpoenadi@Kenneths-MacBook-Air backend % tree -L 3 -d
.
├── controllers
├── helpers
├── middlewares
├── models
└── routes
    ├── auth
    ├── bundle
    ├── email
    ├── formreimburse
    ├── giveaway
    ├── order
    ├── product
    ├── upload
    └── user

15 directories
```

The database uses PostgreSQL to store relational data such as users, products, orders, and transactions. For more flexible data, such as order items, delivery addresses, and Midtrans payment responses, the system uses JSONB fields. This allows the application to store structured but flexible data without creating too many separate tables.

Authentication is handled using Google login and JWT-based sessions. The backend uses middleware to validate the JWT token and attach user information, such as user ID and role, to the request. This structure allows the system to protect user routes and separate access between regular users, admins, and subadmins.

For deployment, the application was deployed on DigitalOcean. The backend service and PostgreSQL database were hosted on a DigitalOcean VPS, while the frontend build served as the client-facing application. This setup allowed the React client to communicate with the Go backend through HTTP API requests, while the backend handled database access and server-side operations.

Overall, the app is structured in a modular and layered way. The frontend focuses on presentation and user interaction, the backend handles business logic and API routing, and the database manages persistent data. This separation makes the project easier to develop, test, and maintain, especially because features like product management, orders, payments, QR pickup, and admin dashboards can be developed independently while still using the same authentication and database system.